ADR-008-mobile-platform-strategy

ADR-008: Mobile Platform Strategy

Status

Accepted

Date

2026-01-08

Context

HelixCode aims to be accessible from multiple platforms including mobile devices. The platform needs mobile support for:

1. **iOS and Android**: Standard mobile platforms
2. **Aurora OS**: Russian mobile operating system
3. **Harmony OS**: Huawei's mobile operating system

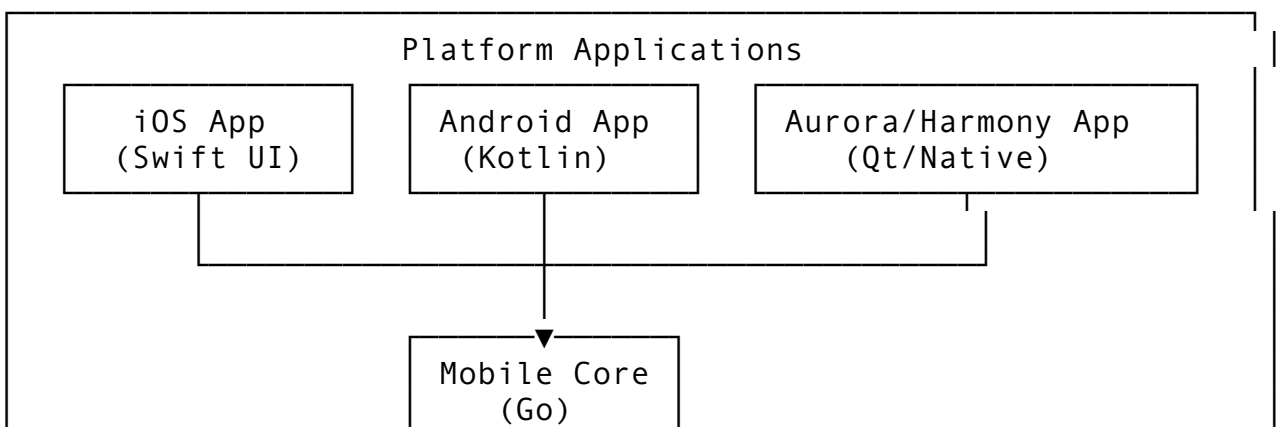
Mobile requirements include: - Task monitoring and management - Worker status visibility - Notification reception - Authentication and session management - Theme customization - Dashboard access

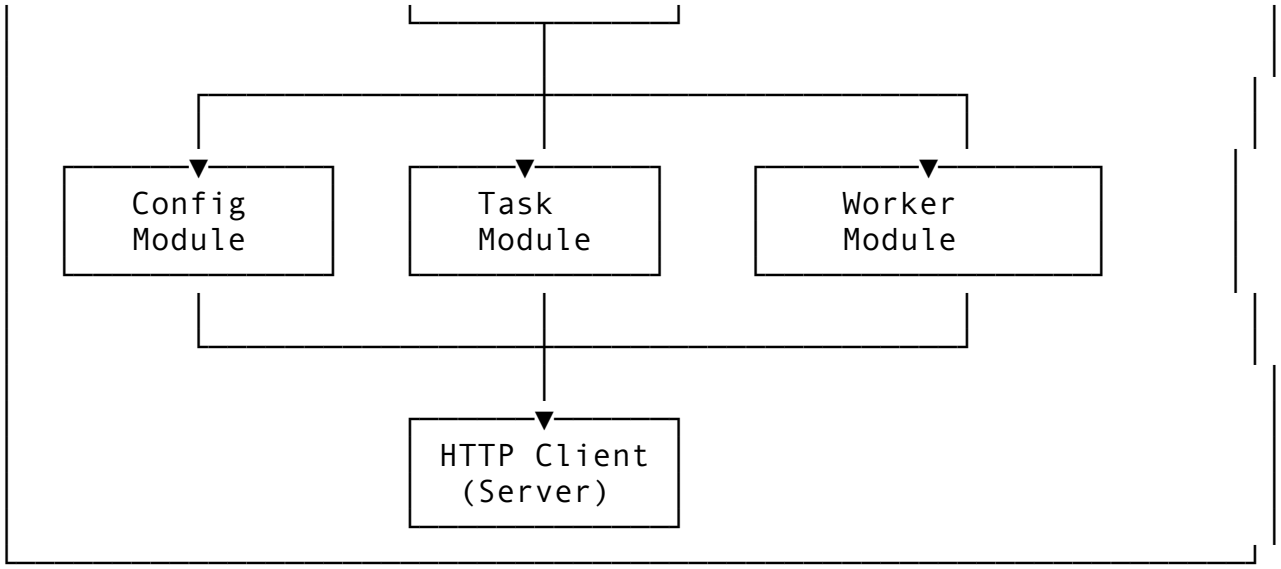
The challenge was enabling mobile access to HelixCode while: - Maximizing code reuse between platforms - Maintaining performance on resource-constrained devices - Handling connectivity issues gracefully - Providing native user experiences - Supporting platform-specific distribution

Decision

We implemented a shared Go core library (`mobile-core`) that compiles to native bindings for each platform using gomobile, with platform-specific UI layers built natively.

Architecture





Shared Mobile Core

Located in `shared/mobile-core/mobile.go`:

```
type MobileCore struct {
    config              *config.Config
    db                  *database.Database
    taskManager         *task.TaskManager
    workerManager       *worker.WorkerManager
    llmProvider         llm.Provider
    notificationEngine *notification.NotificationEngine
    server              *server.Server
    themeManager        *ThemeManager

    // Mobile-specific state
    isConnected  bool
    currentUser  string
    sessionToken string
    mu           sync.RWMutex
}
```

Exported Functions (gomobile bindings)

All public methods are exported for mobile binding:

```
//export NewMobileCore
func NewMobileCore() *MobileCore

//export Initialize
func (mc *MobileCore) Initialize() error

//export Connect
func (mc *MobileCore) Connect(serverURL, username, password
        string) error
```

```go
//export Disconnect
func (mc *MobileCore) Disconnect() error

//export IsConnected
func (mc *MobileCore) IsConnected() bool

//export GetDashboardData
func (mc *MobileCore) GetDashboardData() string  // Returns JSON

//export GetTasks
func (mc *MobileCore) GetTasks() string  // Returns JSON

//export GetWorkers
func (mc *MobileCore) GetWorkers() string  // Returns JSON

//export CreateTask
func (mc *MobileCore) CreateTask(name, description string) string

//export SendNotification
func (mc *MobileCore) SendNotification(title, message,
        notificationType string) string

//export GetTheme
func (mc *MobileCore) GetTheme() string  // Returns JSON

//export SetTheme
func (mc *MobileCore) SetTheme(themeName string) bool

//export Close
func (mc *MobileCore) Close() error
```

Data Exchange Format

All complex data exchanged as JSON strings:

```go
func (mc *MobileCore) getDashboardDataInternal() string {
    data := map[string]interface{}{
        "isConnected": mc.isConnected,
        "user":        mc.currentUser,
        "theme":       mc.themeManager.GetCurrentTheme().Name,
        "stats": map[string]interface{}{
            "tasks":    0,
            "workers":  0,
            "projects": 0,
            "sessions": 0,
        },
    }
    jsonData, _ := json.Marshal(data)
```

```
    return string(jsonData)
}
```

Theme System

Built-in theme support for consistent UI:

```
type Theme struct {
    Name       string
    IsDark     bool
    Primary    string
    Secondary  string
    Accent     string
    Text       string
    Background string
    Border     string
    Success    string
    Warning    string
    Error      string
    Info       string
}
```

Pre-defined themes: - Dark - Light - Helix (brand theme)

Authentication Flow

Mobile authentication supports both real server and mock mode for development:

```
func (mc *MobileCore) connectInternal(serverURL, username,
        password string) error {
    // Attempt server authentication
    resp, err := client.Post(authURL, "application/json",
        authData)
    if err != nil {
        // Fallback to mock authentication for development
        mc.isConnected = true
        mc.currentUser = username
        mc.sessionToken = "mock-token-" + username
        return nil
    }
    // Handle real authentication...
}
```

Platform-Specific Builds

Makefile targets:

```
make mobile         # Build iOS + Android bindings
make aurora-os      # Build Aurora OS client
make harmony-os     # Build Harmony OS client
```

Application Directories

```
applications/
├── terminal-ui/    # TUI application
├── desktop/        # Desktop application (Fyne)
├── aurora-os/      # Aurora OS application
└── harmony-os/     # Harmony OS application
```

Thread Safety

All mobile core methods are thread-safe:

```go
func (mc *MobileCore) getDashboardDataInternal() string {
    mc.mu.RLock()
    defer mc.mu.RUnlock()
    // ... read operations
}

func (mc *MobileCore) connectInternal(...) error {
    mc.mu.Lock()
    defer mc.mu.Unlock()
    // ... write operations
}
```

Consequences

Positive

1. **Code Reuse**: 90%+ logic shared across platforms
2. **Consistency**: Same business logic on all platforms
3. **Maintenance**: Single codebase for core functionality
4. **Performance**: Native Go code, not interpreted
5. **Type Safety**: Go's type system prevents many bugs
6. **Testing**: Core can be tested in Go
7. **Platform Support**: Easily extend to new platforms

Negative

1. **gomobile Limitations**: Not all Go packages work with gomobile
2. **UI Duplication**: Each platform needs native UI code
3. **Build Complexity**: Multiple build targets
4. **Debugging**: Harder to debug across language boundaries
5. **JSON Overhead**: Data serialization/deserialization cost

Neutral

1. **Learning Curve**: Developers need gomobile knowledge
2. **Binary Size**: Go runtime adds to app size

Alternatives Considered

Alternative 1: React Native / Flutter

Description: Use cross-platform mobile frameworks.

Pros: - Single codebase for UI - Large ecosystems - Hot reload for development - Many UI components

Cons: - Different language (JavaScript/Dart) - Another runtime - Performance overhead - Additional dependency

Why Rejected: HelixCode is Go-based. Adding JavaScript/Dart introduces unnecessary complexity and another runtime.

Alternative 2: Pure Native Apps

Description: Build separate native apps for each platform.

Pros: - Best performance - Native UX - Platform-specific features - No abstraction overhead

Cons: - Triplicated business logic - Multiple codebases - Higher maintenance - Team skill requirements

Why Rejected: Duplicating business logic across platforms is error-prone and expensive to maintain.

Alternative 3: PWA (Progressive Web App)

Description: Use web technologies with PWA capabilities.

Pros: - Single codebase - No app store requirements - Web technologies - Easy updates

Cons: - Limited native access - Performance limitations - Offline capabilities limited - Not installable on all platforms

Why Rejected: Mobile-specific features like background tasks and push notifications work better with native apps.

Alternative 4: Kotlin Multiplatform

Description: Use Kotlin Multiplatform for shared code.

Pros: - Android-native integration - Growing iOS support - Shared business logic - Modern language

Cons: - Different language from server - iOS support still maturing - Additional toolchain - Learning curve

Why Rejected: HelixCode is Go-based. Maintaining consistency with gomobile is simpler than adding Kotlin.

Implementation Notes

- Mobile core in `shared/mobile-core/`
- Uses gomobile for binding generation
- JSON for all complex data exchange
- Thread-safe implementation
- Graceful fallback for offline/mock scenarios

Platform Distribution

iOS: App Store distribution **Android**: Google Play Store + APK distribution **Aurora OS**: Aurora Store **Harmony OS**: Huawei AppGallery

Offline Capabilities

The mobile core handles connectivity gracefully: - Connection state tracking - Mock mode for development - Cached data for offline viewing - Queued operations for later sync

Related Decisions

- ADR-005: Authentication System (mobile authentication)
- ADR-001: LLM Provider Interface (mobile can trigger LLM requests)

References

- `/run/media/milosvasic/DATA4TB/Projects/helix_code/helix_code/shared/mobile_core/mobile.go`
- `/run/media/milosvasic/DATA4TB/Projects/helix_code/helix_code/applications/aurora_os/`
- `/run/media/milosvasic/DATA4TB/Projects/helix_code/helix_code/applications/harmony_os/`